Applying Zero-Trust Principles to Kafka: A Performance and Security Trade-off Analysis

Nick Efe Oni

Supervisor: Mahshid Delavar

A report submitted in fulfilment of the requirements for the degree of MEng in Software Engineering

in the

School of Computer Science

April 14, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Nick Efe Oni

Signature: Nick Efe Oni

Date: April 14, 2026

Abstract

an abstract is provided to give a brief overview and summary of the core topics, arguments, and findings . Because it must summarize your entire project—including your final benchmarking results from Chapter 5 and your final conclusions from Chapter 7—you should write the abstract at the very end of the process, once the rest of your dissertation is completely finished.

Contents

1 Introduction **1**

2 Literature Review **3**
2.1 Introduction to Literature Review 3
2.2 Apache Kafka & Distributed Streaming Systems 3
2.3 Performance in Kafka Systems 5
2.4 Security Mechanisms in Kafka 7
2.5 Zero Trust Architecture 8
2.6 Zero Trust in Distributed/Streaming Systems 9
2.7 TLS/mTLS Performance Overhead 9
2.8 Benchmarking Methodologies 9
2.9 Synthesis & Research Gap 9

3 Design, Implementation, and Testing **10**

4 Results and Discussion **11**

5 Evaluation, Conclusions, and Further Work **12**

Appendices **15**

A An Appendix of Some Kind **16**

B Ignore - Templatey Stuff **17**
B.1 Section Template Graph 17
B.2 Section Template Comparison 18
B.3 Summary 18

C Another Appendix **19**

List of Figures

2.1 Simplified Apache Kafka architecture showing producer applications publishing events to cluster of brokers, and consumer applications pulling events from the cluster. 5

B.1 A simple figure in LaTeX. Reproduced from http://tinyurl.com/nqtrlj5 with the permission of the copyright owner. 18

List of Tables

B.1 A simple table in LaTeX. 18

Chapter 1

Introduction

With the evolution of distributed systems [1], event-streaming platforms such as Apache Kafka have been used to connect hundreds of applications in the cloud [2–4], as critical aspects shifted to data consistency, performance optimisation, fault tolerance, and security [5, 6]. Apache Kafka (or Kafka in short) is a popular open-source distributed event-streaming platform [6, 7], known for its performance, low latency, fault tolerance, and high throughput – capable of handling millions of messages per second [3, 4]. These properties make it a critical component in modern data-driven architectures. However, this shift has increased the number of insider threats, as traditional network security was not designed against attacks from within the network [1, 2, 8, 9]. This led industries to increasingly adopt the zero-trust security paradigm, whose core idea means "never trust, always verify" [1, 2, 8–10]. Its implementation includes mutual authentication (mTLS), least-privilege access, continuous monitoring, and data-centric security, which are key to establishing comprehensive protection across distributed environments [1, 2, 8–10].

Despite existing research providing strong theoretical and practical foundations for zero trust in streaming systems [1, 2, 11] and Kafka widely analysed for its properties [3, 7, 12], there is no unified framework and sufficiently systematic empirical evaluation of the performance impact of zero-trust-inspired mechanisms in Kafka-based environments [2, 3, 13]. For example, multiple sources claim that TLS/mTLS encryption introduces significant performance overhead and latency in broader distributed systems [14–16], which is reinforced by Apache Kafka's official documentation stating that enabling TLS/SSL forces cryptographic operations into user space, disabling Kafka's highly efficient sendfile zero-copy optimisation [14, 17, 18]. However, the exact performance degradation remains unquantified.

Therefore, the research question addressed in this dissertation is:

"What is the performance overhead introduced by zero-trust-inspired security mechanisms, particularly mTLS, in high-throughput Kafka-based distributed systems?"

To answer this research question, this work designs and evaluates a configurable Kafka-based

benchmarking framework that supports controlled, one-factor-at-a-time analysis of system and security parameters. Using this framework, performance metrics such as throughput and latency are systematically measured across plaintext, TLS, and mTLS configurations, enabling a rigorous empirical quantification of the trade-offs between security mechanisms and system performance in high-throughput distributed streaming environments.

Chapter 2

Literature Review

2.1 Introduction to Literature Review

This literature review explores the application of modern security paradigms in distributed messaging systems, specifically focusing on the implementation of zero trust in Apache Kafka. It provides the context behind the previously identified research gap: the lack of empirical quantification regarding the trade-offs between security mechanisms and system performance in Kafka-based high-throughput environments.

Organised into eight themes, this review initially establishes the architecture, performance behaviour, and native optional security mechanisms of Apache Kafka. The focus then shifts to the principles of zero trust and the challenges of applying these concepts to distributed systems, using Kafka as an application context. It then concludes by critically analysing the existing literature on the performance overhead of cryptographic protocols, particularly TLS and mTLS, and by evaluating the benchmarking methodologies used to measure throughput and latency in distributed systems.

2.2 Apache Kafka & Distributed Streaming Systems

Distributed streaming systems, such as Apache Kafka, are engineered to be fault-tolerant [6, 12, 17], have high-availability [6, 12], and handle the storage and parallel processing of massive streams of data [6, 12, 17], while ensuring data consistency and reliability [3, 12].

Apache Kafka consists of servers (brokers) and clients (producers/consumers) that communicate via a high-performance TCP network protocol, which contributes to Kafka's performance [6, 12]. Producers are client applications that publish (write) events to brokers, whereas consumers are those that subscribe to (read and process) these events by pulling from the brokers [6,12]. A simple illustration of this can be found in Figure 2.1. Thus, brokers act as an intermediary cluster of servers between producers and consumers [6, 12]. These decoupling and pull-based consumption mechanisms contribute to Apache Kafka's fault

tolerance and high throughput [6,17]. Consumer applications can fetch large data batches at their own maximum processing capacity without being overwhelmed by the data production rate [17]. Moreover, events are organised and durably stored in "topics", which are implemented as append-only, immutable, time-ordered logs on the disk [3,6,17]. Another key design element of Apache Kafka is the subdivision of topics into partitions. By subdividing topics into partitions distributed across multiple brokers, massive horizontal scalability and the storage and parallel processing of massive streams of data are achieved [6, 12, 17]. Despite the benefits, there are also trade-offs and drawbacks. Kafka's performance is affected by batching, acknowledgement semantics, and broker-level thread and quota configuration, which introduce explicit throughput-latency-durability trade-offs [3, 17–19]. In addition, maintaining consistency across replicas and coordinating partition leadership can impose non-trivial overhead, particularly under sustained high-throughput workloads and inappropriate configurations [3, 17, 18]. In essence, Apache Kafka is a widely adopted distributed streaming system with multiple architectural features and mechanisms designed to support data consistency, reliability, low latency, and high throughput [3,6,12,17]. However, these benefits come with trade-offs and complexities, as Apache Kafka's performance emerges not solely from its architectural features and mechanisms, but also from the selected configuration, coordination costs and the deployment environment [3, 17–20]. This has important implications for the integration of security protocols such as TLS and mTLS, which introduce additional communication and computational overhead and therefore necessitate careful evaluation within Kafka's distributed architecture [14, 18–20].

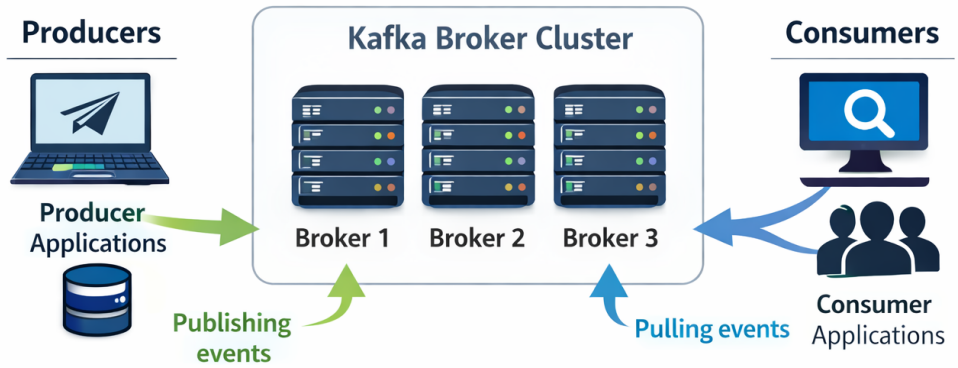

Figure 2.1: Simplified Apache Kafka architecture showing producer applications publishing events to cluster of brokers, and consumer applications pulling events from the cluster.

2.3 Performance in Kafka Systems

The performance of Apache Kafka arises from key architectural choices. For example, bypassing traditional user-space memory management and $O(\log N)$ random-access data structures, such as B-Trees, in favour of simple constant-time $O(1)$ append-only logs [3,6,17]. Furthermore, it heavily relies on the operating system's page cache and aggressively employs zero-copy I/O mechanisms, specifically the 'sendfile' system call, to transfer data directly from the kernel to the network socket [17]. By offloading memory management to the OS and using $O(1)$ sequential disk operations, Kafka maintains maximum performance and avoids unnecessary data movements [6, 12, 17]. This dependence on OS-level optimisations makes the system incredibly fast, but also highly sensitive to user-space interventions introduced by security mechanisms such as TLS and mTLS [14–18].

Additionally, Kafka's speed is driven by data-flow mechanisms such as parallel partitioning, aggressive batching, and decoupled pull-based consumption [3, 6, 12, 17]. Topics are divided into partitions spread across multiple brokers, enabling massive horizontal scalability by allowing client applications to read and write data in parallel [6, 12, 17]. Producers amortise

network round-trip overhead through asynchronous sends, utilising specific configurations to group individual events into dense, compressed payloads [3, 12, 17, 19]. Concurrently, consumers operate on a pull-based model utilising lightweight offset tracking, allowing them to fetch these large, compressed batches at their own maximum capacity without overwhelming the broker [6,7,12,17,20]. Due to these mechanisms, the computational burden of decompression shifts to the consumer, drastically reducing network bandwidth saturation and the CPU load of the brokers [3,12,17,19,20]. Consequently, Kafka's native performance heavily relies on keeping these network payloads perfectly tuned and lightweight, conditions that are fundamentally disrupted when complex user-space cryptography is introduced to the process [14–20].

Moreover, Kafka configuration choices introduce explicit throughput-latency-durability trade-offs [3, 12, 17–20]. Producers can be configured to require full consensus from all in-sync replicas or push events asynchronously [3, 12, 17, 19]. Although the former guarantees durability, it significantly increases latency and decreases throughput, thus degrading raw performance [3, 12, 17–19]. Similarly, enforcing Exactly-Once Semantics (EOS) to prevent duplicate messages further increases latency by requiring strict state tracking and transactional coordination across the cluster [3, 12, 17–19]. Overall, these trade-offs demonstrate that Apache Kafka's performance is not a static absolute, but a highly configurable spectrum in which the overhead of mandatory operational constraints, such as those required for highly secure zero-trust environments, must be carefully benchmarked [3,14–20].

Furthermore, according to a queueing-theoretic model applied to publish-subscribe systems, the average latency (L) of a message is mathematically defined by the processing time (τ) and system utilisation (U) as:

$$L = \frac{\tau}{1 - U} \tag{2.1}$$

[7,12,16]. Any configuration or architectural shift that increases processing time (τ) pushes CPU utilisation (U) closer to 1.0, causing latency to grow non-linearly rather than linearly as the system approaches saturation [3, 7, 12, 16]. This theoretical relationship provides a foundation for understanding the identified research gap: because Zero Trust mechanisms such as Mutual TLS (mTLS) force operations into user-space, their performance overhead cannot be assumed to be linear, necessitating rigorous, context-specific empirical evaluation in high-throughput Kafka systems [3, 7, 14–18]. This is because, by forcing operations into user-space, it effectively disables or limits zero-copy optimisations and significantly increases τ via complex cryptography [7, 14–18].

Ultimately, Kafka's performance, while capable of achieving high throughput, is highly sensitive to both internal client architecture and the underlying realities of system utilisation [3, 7, 16, 18–20]. This highlights the need to examine how security mechanisms

are implemented within Kafka systems, particularly those that introduce additional computational and communication overhead. Understanding these mechanisms is therefore essential before evaluating their impact on performance in high-throughput environments [3, 14–20].

2.4 Security Mechanisms in Kafka

To protect sensitive data streams against both external and internal threats, Kafka provides a native security toolkit that includes encryption and authentication [6, 13, 18–20]. However, while these controls successfully establish a secure boundary, their activation fundamentally alters the mechanics of Kafka's data flow, introducing systemic complexities that directly conflict with the platform's baseline performance design [14–20]. These gaps directly motivate the research presented in this dissertation.

For instance, encryption in transit protects the confidentiality of data payloads across distributed environments, preventing eavesdropping and Man-in-the-Middle attacks [1, 9, 13, 15, 18–20]. This is implemented through Transport Layer Security (TLS) via specific listener protocols, allowing administrators to precisely dictate cryptographic strength using security configurations such as '*ssl.enabled.protocols*' and '*ssl.cipher.suites*' [13, 15, 18–20]. By enabling these security configurations, Apache Kafka's performance capabilities significantly degrade due to a compounding bottleneck [3, 13–15, 17, 18]. This bottleneck emerges from the heavy mathematical overhead of encrypting and decrypting every payload, and the loss of zero-copy I/O optimisations, since TLS/SSL libraries are required to operate in user space by design [14–18]. Despite the literature possessing a clear critique regarding this specific architectural clash and the analysis of TLS overhead in traditional stateless web servers, empirical research explicitly quantifying this degradation within Kafka and its specific configurations remains sparse [3, 13–18].

On the other hand, there is mTLS, which is enforced through the broker's *ssl.client.auth* configuration, allowing the use of cryptographically signed $X.509$ certificates to verify identities between brokers, consumers, and producers [13, 15, 18–20]. However, because Apache Kafka's distributed design relies heavily on internal replication, mTLS must also be applied to all internal broker-to-broker traffic, substantially increasing the cryptographic workload across the entire system and significantly reducing performance [9, 14–18]. Moreover, if thousands of clients attempt to reconnect simultaneously, repeated TLS/mTLS handshakes create an authentication surge that consumes broker resources and interferes with data-plane performance [1, 2, 14–18].

In conclusion, while Apache Kafka has native security measures, enabling them causes heavy computational costs that are insufficiently explored across the literature, as evident by the sparse, often incomplete and inconsistently reported empirical evaluations of the

configurations used and trade-offs [3, 7, 13–20].

2.5 Zero Trust Architecture

The limitations of native security mechanisms in distributed environments highlight the necessity for a more comprehensive security paradigm, leading to the widespread adoption of Zero Trust Architecture (ZTA) [1, 2, 5, 8, 9]. Unlike traditional perimeter-based models that assume internal networks are secure, ZTA operates on the foundational principle of "never trust, always verify," eliminating implicit trust entirely [1, 2, 8–10]. Instead, security is maintained through continuous verification, the enforcement of least-privilege access, and strict identity-based control [1, 2, 8–10]. This paradigm shift ensures that security is proactively applied at every interaction rather than merely at the network boundary, an approach that is increasingly critical in modern, highly decentralised cloud environments [1, 2, 8, 9].

To enforce this continuous verification, ZTA relies on a combination of dynamic policies and structural components rather than a single monolithic technology [1, 8–10]. At its core, ZTA necessitates context-aware authentication and fine-grained access control models, frequently implemented through logical components such as Policy Decision Points (PDP) and Policy Enforcement Points (PEP) [1, 8–10]. Furthermore, a fundamental requirement for establishing these secure, segmented interactions is the mandatory encryption of all communication channels [1, 8–10]. Consequently, cryptographic protocols such as TLS and mTLS become foundational mechanisms in many Zero Trust implementations rather than optional add-ons [1, 8, 9, 15].

While ZTA provides robust protection against both external and insider threats, analytical studies reveal that its implementation introduces significant operational complexities [1, 8–10]. Transitioning to a zero-trust model requires major infrastructure restructuring, complex identity management, and continuous monitoring, making the paradigm resource-intensive [1, 2, 5, 8–10]. This introduces a critical architectural tension: the strict, continuous cryptographic verification demanded by zero trust consumes substantial computational resources, threatening to disrupt the delicate performance balance of latency-sensitive systems [1, 7–9, 14–17].

2.6 Zero Trust in Distributed/Streaming Systems

2.7 TLS/mTLS Performance Overhead

2.8 Benchmarking Methodologies

2.9 Synthesis & Research Gap

Chapter 3

Design, Implementation, and Testing

This is where you explain how you are going to solve the problem. To get top marks, you must provide an in-depth analysis of the problem area, outline clear project requirements, and present a precise, methodical design. For your project, this will likely be your methodology chapter outlining your Kafka testing environment. You must thoroughly describe how you built your experimental system and how you conducted your tests. The rubric specifically looks for a comprehensive discussion of your testing processes

Chapter 4

Results and Discussion

This is where you present your findings (e.g., your throughput and latency data). To reach the 70+ band, your results must be presented in a logical order alongside a "perceptive and critical discussion" of what the data actually means

Chapter 5

Evaluation, Conclusions, and Further Work

Your final chapter must evaluate how well you satisfied your original objectives, provide an insightful summary of your conclusions, and discuss appropriate directions for future research

Bibliography

[1] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and application of zero trust security: A brief survey," *Entropy*, vol. 25, no. 12, p. 1595, 2023.

[2] H. Bhoite, "Zero-trust architecture in streaming dataflows," https://www.authorea.com/doi/full/10.36227/techrxiv.175303721.12297807, accessed: 2026-03-26.

[3] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," in *2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2025, pp. 612–617.

[4] "Powered by," https://kafka.apache.org/powered-by/, accessed: 2026-03-24.

[5] A. Kumar, "Distributed systems in modern enterprise architecture: Challenges and solutions," *IJSAT*, vol. 16, no. 1, p. 2447, 2025.

[6] "Introduction," https://kafka.apache.org/42/getting-started/introduction/, accessed: 2026-03-24.

[7] A. Sundbom, "Parallel kafka producer applications: Their performance and its limitations," Master's thesis, KTH Royal Institute of Technology, 2023.

[8] V. A. Vora, "Demystifying zero trust security: The no-trust network paradigm," *Journal of Computer Science and Technology Studies*, vol. 7, no. 3, pp. 141–148, 2025.

[9] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (zta): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.

[10] M. Tsai, S. Lee, and S. W. Shieh, "Strategy for implementing of zero trust architecture," *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 93–100, 2024.

[11] A. A. Süzen and O. Ceylan, "The advantages and implementation challenges within the scope of the basic principles of transition to zero trust architecture," *International Journal of 3D Printing Technologies and Digital Industry*, vol. 8, no. 3, pp. 416–427, 2024.

[12] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.

[13] J. Meka, "Financial services cloud transformation: Securing sensitive data in kafka event streams," *Journal of Computer Science and Technology Studies*, vol. 7, no. 4, pp. 1023–1028, 2025.

[14] C. Coarfa, P. Druschel, and D. S. Wallach, "Performance analysis of tls web servers," *ACM Transactions on Computer Systems*, vol. 24, no. 1, pp. 39–69, 2006.

[15] C. H. Jack, S. K. Teck, L. T. Ming, and D. Y. Hong, "An overview analysis of authentication mechanism in microservices-based software architecture: A discussion paper," in *2023 IEEE 8th International Conference on Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 1–6.

[16] M. Iorio, F. Risso, and C. Casetti, "When latency matters: Measurements and lessons learned," *SIGCOMM Computer Communication Review*, vol. 51, no. 4, pp. 2–13, 2021.

[17] "Design," https://kafka.apache.org/42/design/design/, accessed: 2026-03-24.

[18] "Broker configs," https://kafka.apache.org/42/configuration/broker-configs/, accessed: 2026-03-26.

[19] "Producer configs," https://kafka.apache.org/42/configuration/producer-configs/, accessed: 2026-03-26.

[20] "Consumer configs," https://kafka.apache.org/42/configuration/consumer-configs/, accessed: 2026-03-26.

Appendices

Appendix A

An Appendix of Some Kind

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.

Appendix B

Ignore - Templatey Stuff

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. [**?**] Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum.

$$M = \frac{1}{T}\sum_{t=1}^{T} e(t)/\max_{t}[e(t)] \tag{B.1}$$

Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.

B.1 Section Template Graph

Content: Explain how microservices talk to each other.

Figure: Insert a diagram of a "Producer -¿ Broker -¿ Consumer" architecture here.

See Figure B.1.

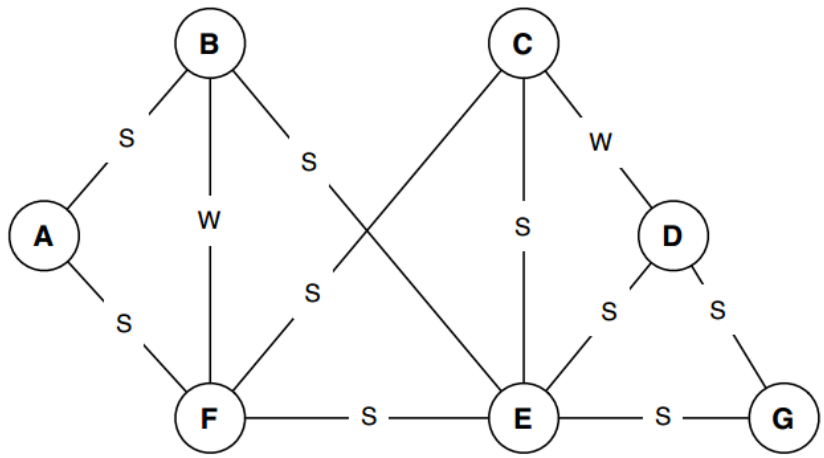

Figure B.1: A simple figure in LaTeX. Reproduced from http://tinyurl.com/nqtrlj5 with the permission of the copyright owner.

B.2 Section Template Comparison

Content: Lorem

Table: Use the template's table code to compare them (e.g., Column A=Lorem, B=Inps, C=Lode).

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Table B.1: A simple table in LaTeX.

B.3 Summary

Content: "Existing benchmarks are outdated. This project fills that gap."

Appendix C

Another Appendix

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.